

EMBEDDED SYSTEM WITH IOT

EXPERIMENT NO. 4

IOT-BASED IRRIGATION SYSTEM

Name: *Raphael S. Nano*

Course/Section: *CPE162P / CI*

Date of Performance: *March 8-9, 2024*

Date of Submission: *March 11, 2024*

Ma'am Cyrel Manlises

Instructor

DISCUSSION

Soil quality is critical for plant health, and variables that increase soil quality include sunlight, nutrients, and moisture. The soil's moisture content might vary depending on the plant. Some may require a lot of water, while others can thrive with little to none. With the usage of IoT, a system for watering plants would be more convenient and efficient, with soil moisture detected in real time and visible to the user on another device. Hence, for this experiment, an IoT-based Irrigation System was developed using ESP8266 as the controller, soil moisture sensor as the main input, LCD for the displaying of values, and servo motor for the pouring of water.

For experiment 4, the objectives are as follows:

- 1. To demonstrate how an IoT based Irrigation System works.*

2. To be able apply what you have learned in logic circuits and switching theory, microprocessors, CPE160P, and CPE161P.

3. To be able to apply embedded system with IoT using Arduino kit

4. To be able to use ThingSpeak for the data collection and analysis.

The materials involved in constructing the IoT-based Irrigation System are the ESP8266 WiFi module, soil moisture probe and LM393 module, liquid crystal display, and SG90 servo motor.

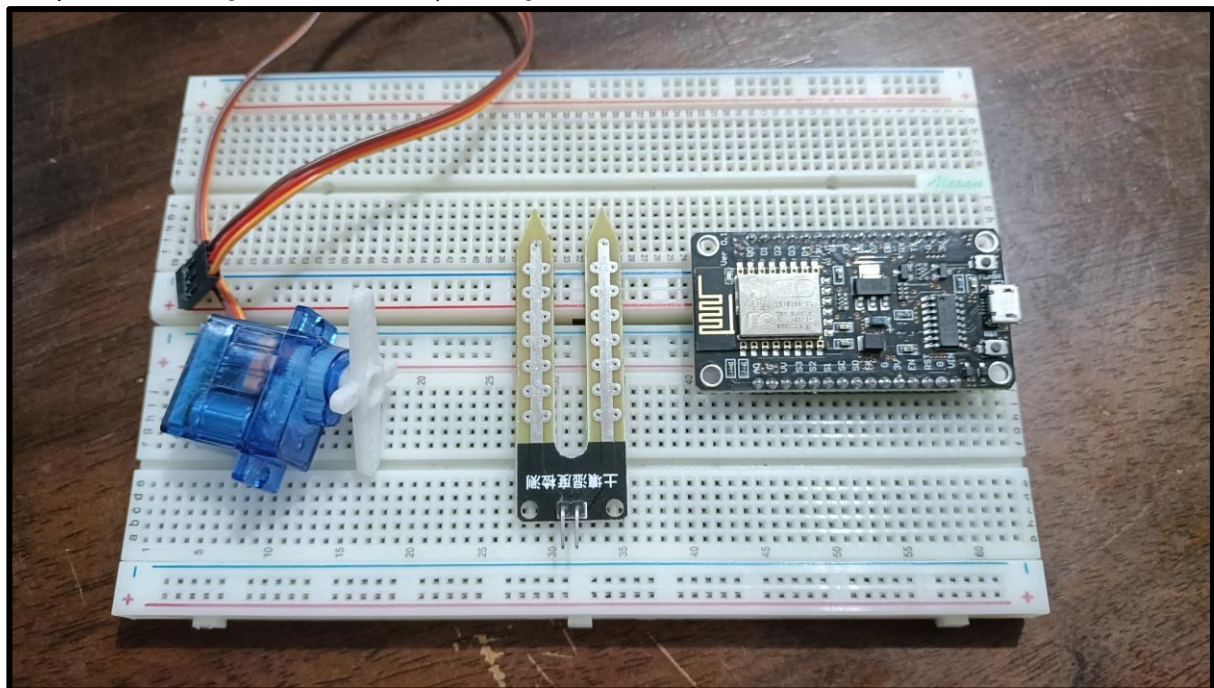


Figure 1. Major Components for the Experiment

For the external libraries, ESP8266WiFi.h and ThingSpeak.h are some of the separate libraries included. Secrets.h is a file with definitions for ESP8266WiFi.h. For the LCD, LiquidCrystal_I2C.h was utilized, and Servo.h for the servo motor.

```
#include "Servo.h"
#include <LiquidCrystal_I2C.h>
#include <String.h>
#include <ESP8266WiFi.h>
#include "ThingSpeak.h"
#include <Arduino.h>
```

Figure 2. External Libraries Included

The primary aim of experiment 4 is to detect the change in moisture level in the soil and controls the flow of water accordingly with a motor. ThingSpeak will also be utilized for data collection and analysis, including value graphing.

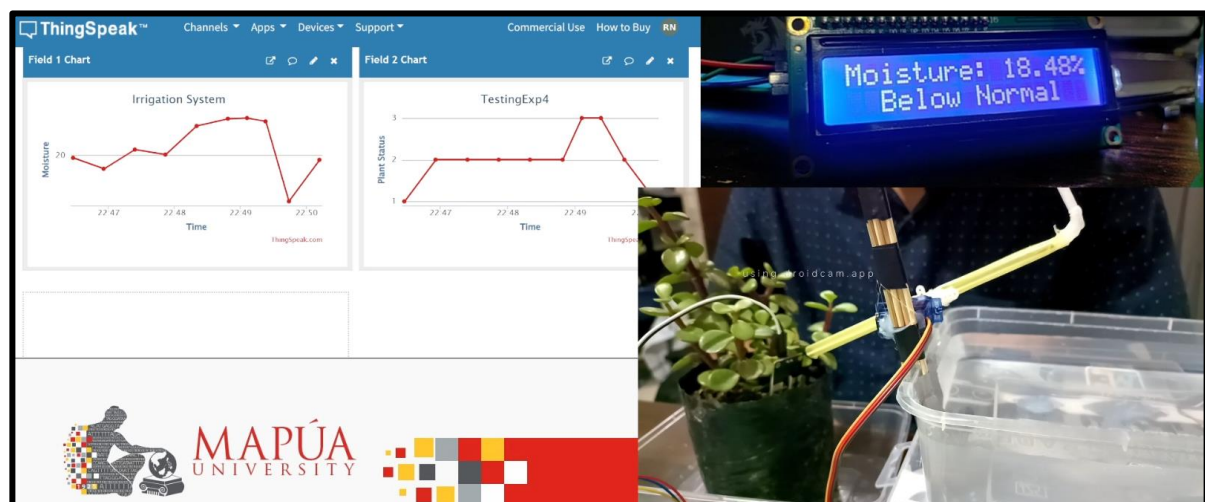


Figure 3. Sample Out put of Experiment 4

The main components of this experiment are the detection of soil moisture, displaying of values in the LCD, and rotation of servo motor to pour water. With the use of readMoist() function, the soil moisture sensor detects the moisture percentage. The lcd.print() function is then utilized in order to output the value of the moisture. Then, for the servo motor, a separate function was created named the servoOp(), wherein a straw connected to the servo would scoop water and pour it on the plant.

```
void readMoist() {  
  moisture = (100.00 - ((analogRead(sensor) / 1023.00) * 100.00));  
  Serial.print("Soil Moisture(in Percentage) = ");  
  Serial.print(moisture);  
  Serial.println("%");  
}
```

Figure 4. Soil Moisture Sensor Main Function

```
void servoOp() {  
  servo.write(180);  
  delay(3000);  
  for (int i = 180; i > 0; i--) {  
    servo.write(i);  
  }  
  delay(5000);  
}
```

Figure 5. Servo Motor Function

The same code was utilized in establishing the Wi-Fi connection. It provides information to the user via the Serial Monitor, indicating the status of the connection attempt.

```
if (WiFi.status() != WL_CONNECTED) {  
  Serial.print("Attempting to connect to SSID: ");  
  Serial.println("fh_dbe600");  
  while (WiFi.status() != WL_CONNECTED) {  
    WiFi.begin(ssid, pass);  
    Serial.print(".");  
    delay(5000);  
  }  
  Serial.println("\nConnected.");  
}
```

Figure 6. WiFi Function

ThingSpeak updates its fields every 15 seconds. In this situation, there are two fields: moisture percentage and plant status. The values are updated using the ThingSpeak's writeFields() function.

```
if ((millis() - timeStart) >= 15000) {  
  ThingSpeak.setField(1, moisture);  
  ThingSpeak.setField(2, plantStat);  
  
  int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);  
  if (x == 200) {  
    Serial.println("Channel update successful.");  
  } else {  
    Serial.println("Problem updating channel. HTTP error code " + String(x));  
  }  
  timeStart = millis();  
}
```

Figure 7. Thingspeak Function

In the main loop, the identification of the moisture percentage is every 3 seconds. For this experiment, there are 4 conditions: Too Dry, Below Normal, Normal, and Drowning. The moisture content for a plant that is too dry is less than 10%. 10% to 30% for below normal, 30% to 50% for normal, and greater than 50% for drowning. Only the Too Dry and Below Normal conditions have the servoOp() function since these types of plants need water.

```
if (moisture < 10) {  
  lcd.clear();  
  lcd.setCursor(0, 0);  
  lcd.print("Moisture: ");  
  lcd.setCursor(10, 0);  
  lcd.print(moisture);  
  lcd.print("%");  
  lcd.setCursor(5, 1);  
  lcd.print("Too Dry");  
  servoOp();  
  plantStat = 1;  
}
```

```
} else if ((moisture > 10) && (moisture < 30)) {  
  lcd.clear();  
  lcd.setCursor(0, 0);  
  lcd.print("Moisture: ");  
  lcd.setCursor(10, 0);  
  lcd.print(moisture);  
  lcd.print("%");  
  lcd.setCursor(2, 1);  
  lcd.print("Below Normal");  
  servoOp();  
  plantStat = 2;  
}
```

```
} else if ((moisture >= 30) && (moisture < 50)) {  
  lcd.clear();  
  lcd.setCursor(0, 0);  
  lcd.print("Moisture: ");  
  lcd.setCursor(10, 0);  
  lcd.print(moisture);  
  lcd.print("%");  
  lcd.setCursor(5, 1);  
  lcd.print("Normal");  
  plantStat = 3;  
}
```

```
} else if (moisture >= 50) {  
  lcd.clear();  
  lcd.setCursor(0, 0);  
  lcd.print("Moisture: ");  
  lcd.setCursor(10, 0);  
  lcd.print(moisture);  
  lcd.print("%");  
  lcd.setCursor(4, 1);  
  lcd.print("Drowning!");  
  plantStat = 4;  
}
```

Figure 8. Main Loop Function

RESULTS

According to Rutherford, Groves, and Sajeva (2018), although the 4 plants are different in appearances, most succulent plants do not need that much water. Hence, little moisture is required for the experiment, and that little moisture could be attributed to Normal, which is tested to be 30 to 50 percent.

The IoT-based Irrigation System consisted of various conditions in order to fully test the system. For this experiment, there are 4 conditions: 1 for Too Dry, 2 for Below Normal, 3 for Normal, and 4 for Drowning. The moisture percentage of the different plants were determined using a soil moisture sensor, and the value was also posted in ThingSpeak, as well as the plant status. As seen in the graphs and LCD samples below, the Too Dry plant condition required a bit of time for it to be normal, wherein approximately four servoOp() calls were done,

meaning that the servo motor scooped water four times to make it normal.

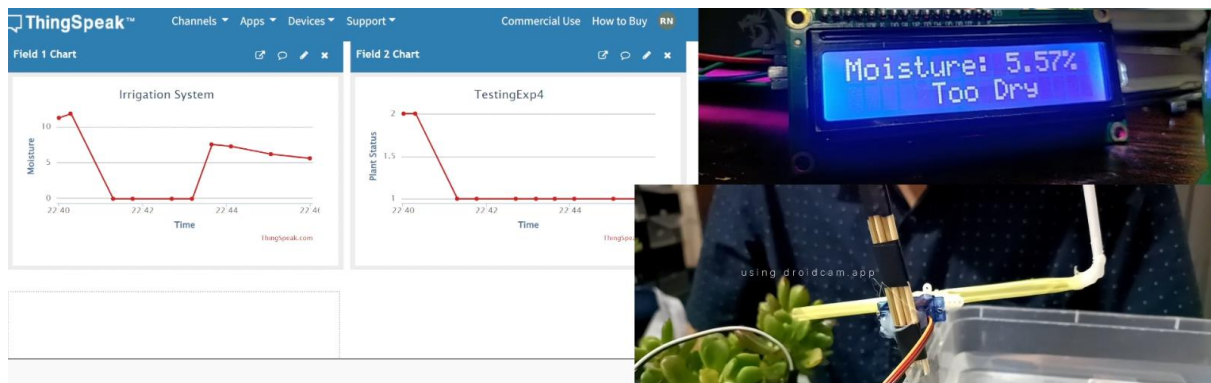


Figure 9. Too Dry condition

For Below Normal, it only required three times.

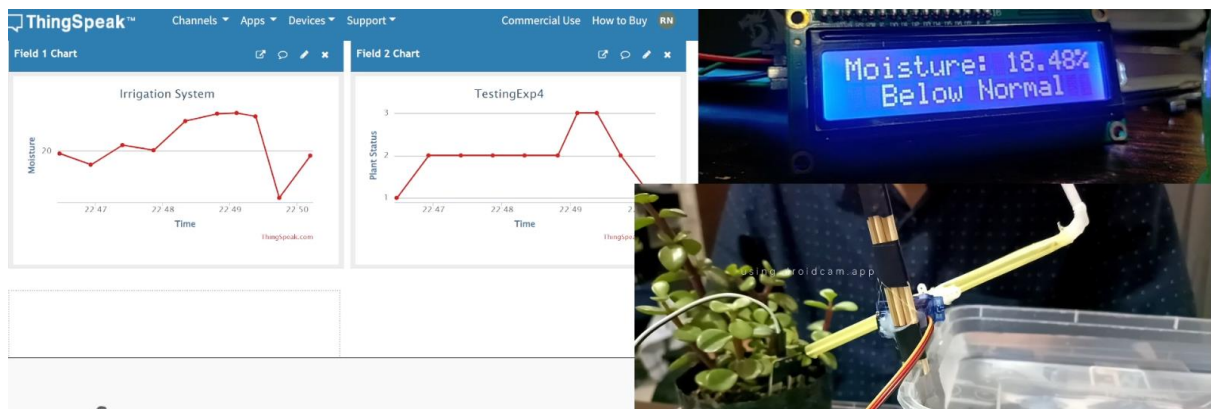


Figure 10. Below Normal condition

Of course, the Normal and Drowning conditions did not trigger the servo motor. All of the values were displayed in ThingSpeak and on the LCD. Hence, all of the required conditions were tested successfully and the outputs are as expected.

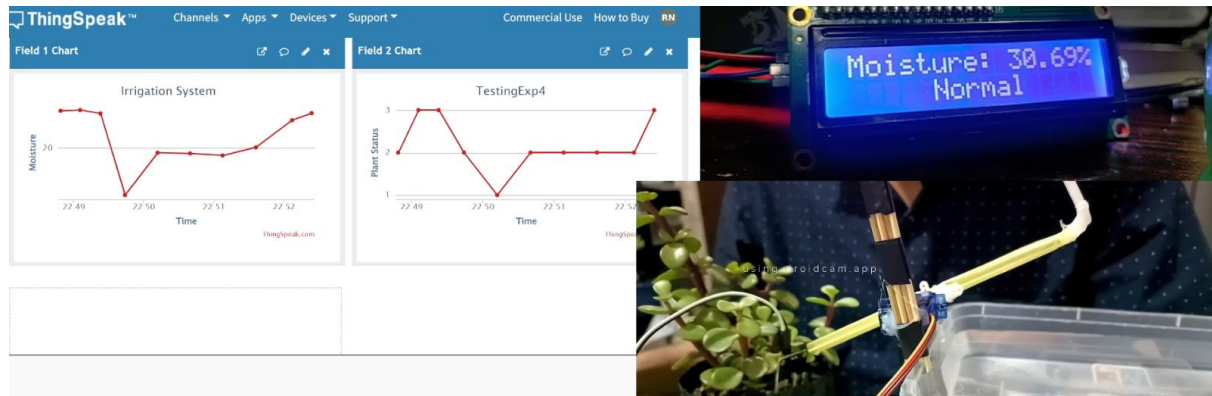


Figure 11. Normal condition

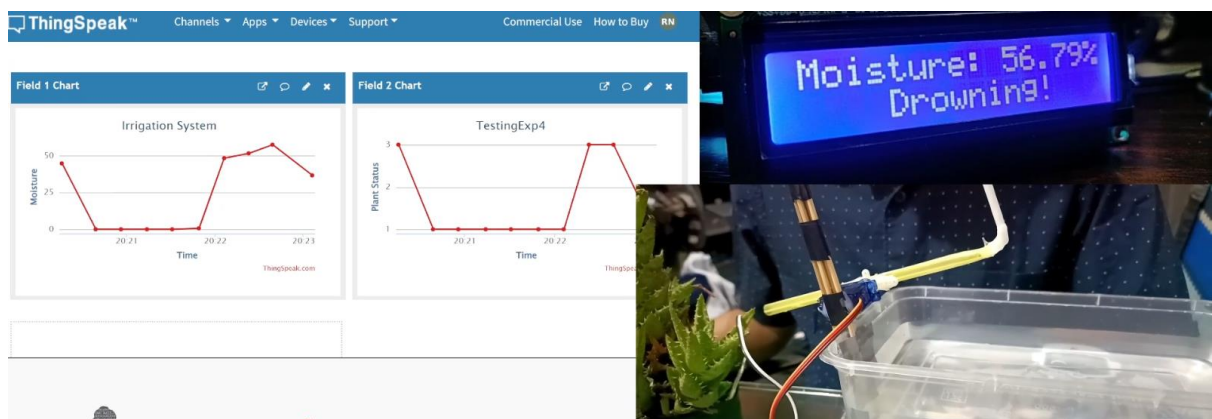


Figure 12. Drowning condition

CONCLUSION

Ultimately, the IoT-based Irrigation System uses the readMoist(), lcd.print(), and servoOp() functions to read the soil moisture percentage every 3 seconds, print the moisture value on the LCD, and activate the servo motor if the condition is too dry or below normal. Every 15 seconds, ThingSpeak updates, displaying both the moisture value and the plant status. The CPE160P and CPE161P courses were explicitly implemented by connecting the ESP8266 with ThingSpeak using Arduino IDE codes and applying the LCD, soil moisture sensor, and servo motor libraries, resulting in the second objective. The ThingSpeak effectively shows moisture percentage and plant status every 15 seconds, which was accomplished by utilizing the writeFields() function. As a result, the IoT-based Irrigation System was successfully developed, with accurate values and clear outputs in the servo motor, LCD, and ThingSpeak.